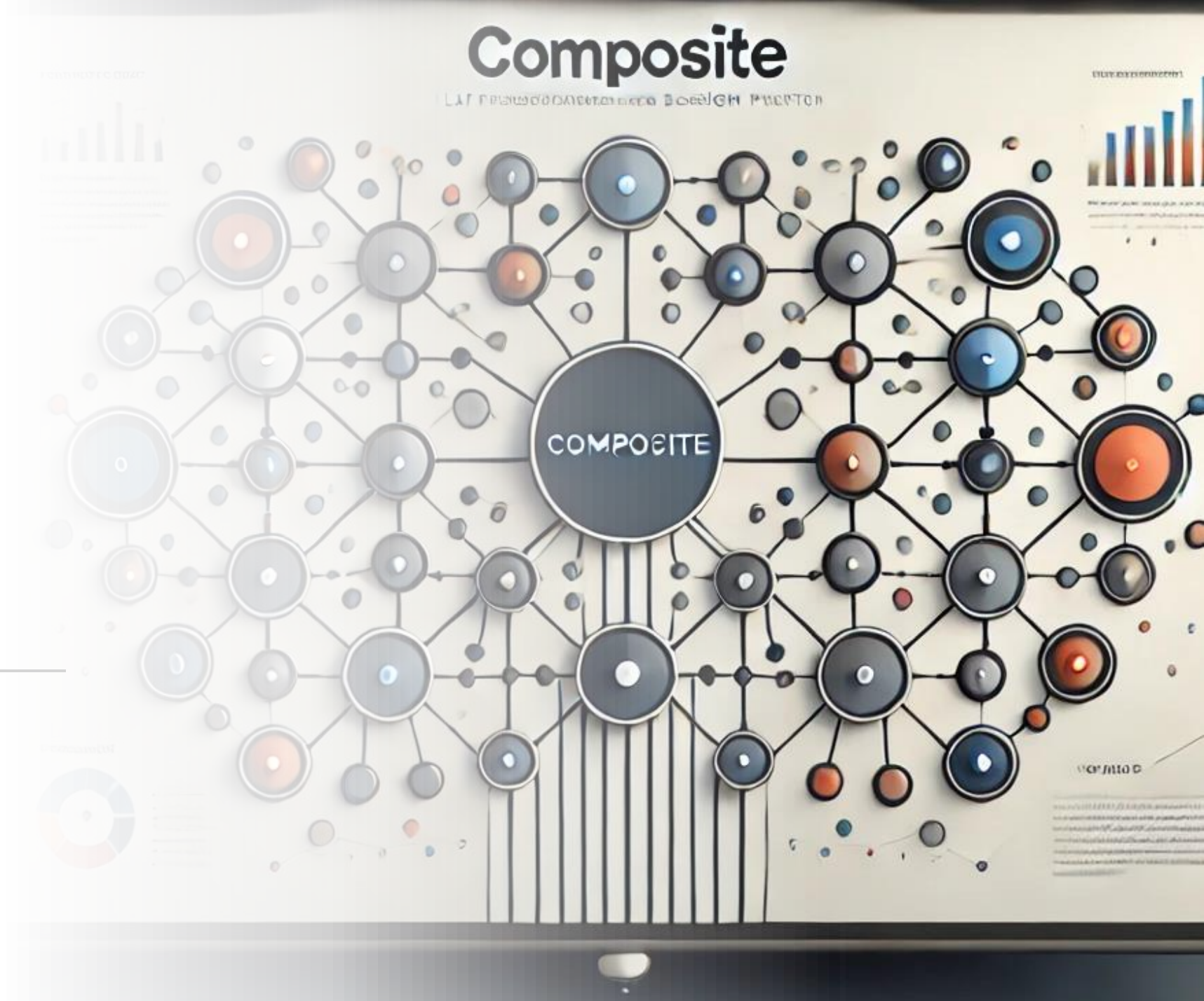


Composite

Padrões de Projeto Estrutural II

Prof. Me Jefferson Passerini



O padrão **Composite** permite a composição de objetos em estruturas de árvore para representar hierarquias parte-todo. Com esse padrão, os clientes podem tratar objetos individuais ou composições de objetos de maneira transparente e uniforme.

Aplicabilidade

- Quando é necessário representar uma hierarquia parte-todo de objetos.
- Quando é preciso que os clientes possam ignorar a diferença entre composições de objetos e objetos individuais, ou seja, os clientes devem tratar todos os objetos na estrutura composta de maneira uniforme.

Componentes

- **Cliente:** O cliente utiliza a interface Componente para manipular os objetos da composição.
- **Componente:** Define uma interface para todos os objetos da composição, o que inclui tanto os compostos quanto as folhas. Pode definir uma interface para acessar o pai de um componente na estrutura recursiva e a implementa, se apropriado.
- **Folha:** Define o comportamento dos componentes primitivos, ou seja, que não possuem filhos. Isto é feito implementando as operações que o Componente aceita.
- **Composite:** Também implementa as operações do Componente, o papel do composite é definir o comportamento dos componentes que possuem filhos. O composite armazena seus filhos

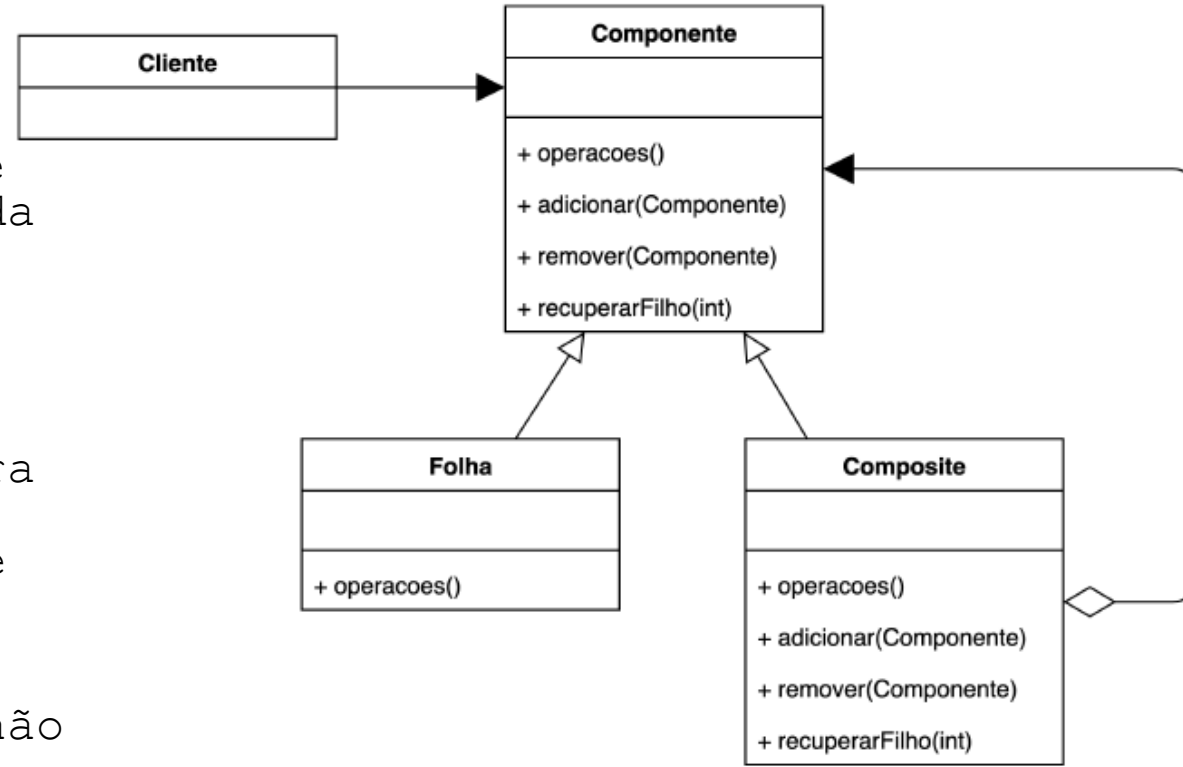


Diagrama de Classes

Motivação (Por que utilizar?)

- A imagem ao lado ilustra uma estrutura de dados em árvore.
- Os elementos com filhos são chamados de **Nós** e os elementos sem filhos são chamados de **folhas**.
- Há momentos em que um software precisa manipular uma estrutura de dados em árvore, e para evitar complexidade, é necessário tratar os nós e as folhas de maneira uniforme.

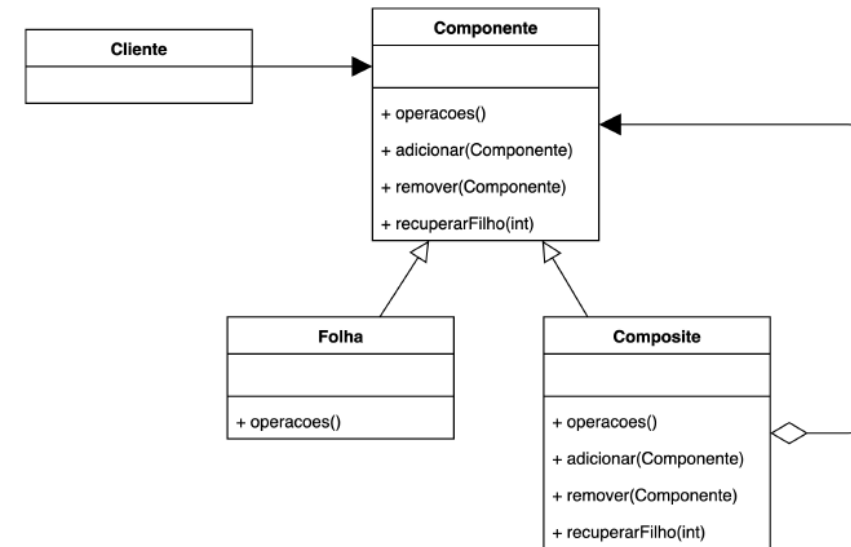
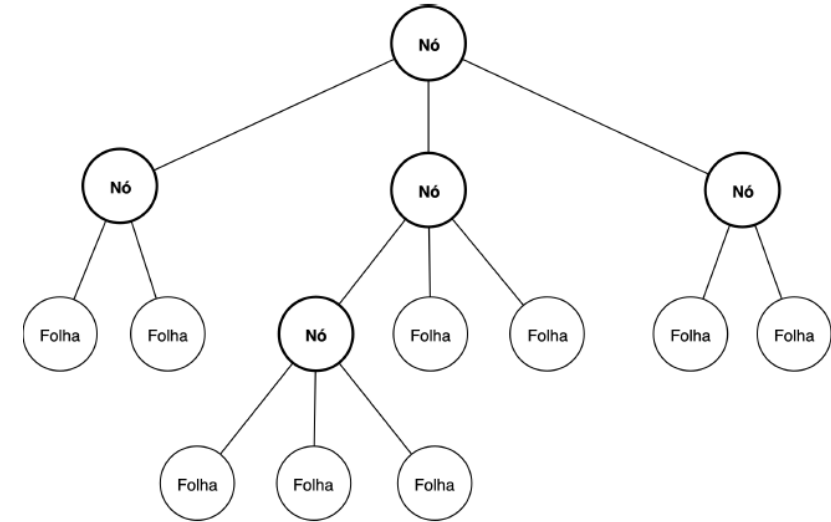
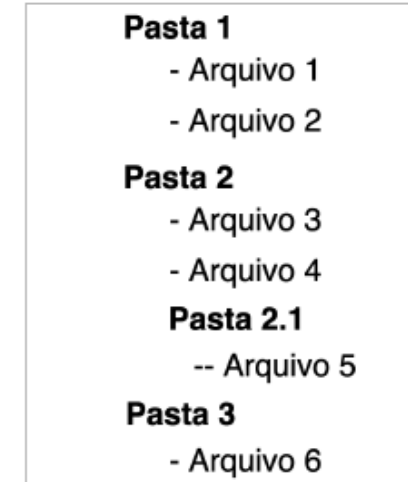


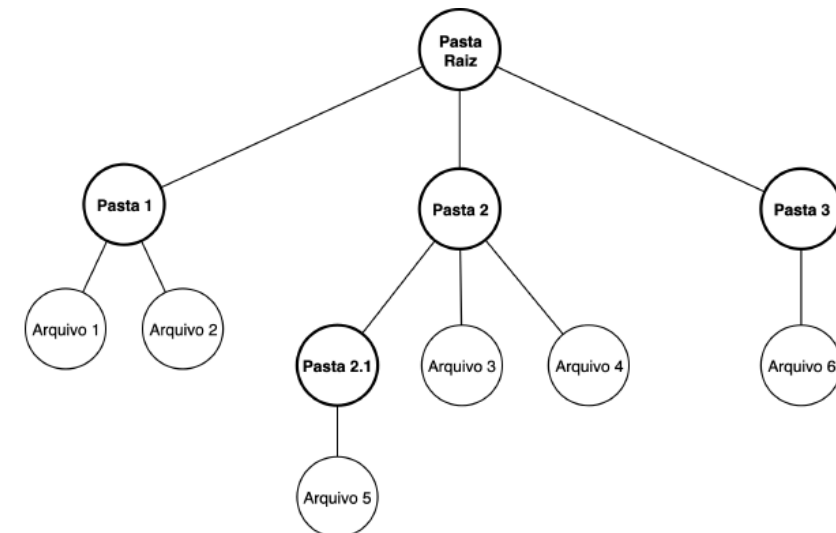
Diagrama de Classes

Motivação (Por que utilizar?)

- Considere, por exemplo, um sistema de arquivos. Pode ser uma estrutura em árvore que contém nós (pastas), e folhas (arquivos). Um objeto de pasta geralmente contém um ou mais objetos de arquivo ou outras pastas (subpastas), portanto, uma pasta é um objeto complexo e um arquivo é um objeto simples.
- Considere a hierarquia do sistema de diretórios acima, nosso objetivo a nível de explicação é organizar essa estrutura e tornar possível a inclusão, remoção e consulta de arquivos, tal consulta pode ser a nível geral, por pastas ou subpastas.
- A imagem ao lado exibe como esse mesmo sistema de arquivos ficaria organizado como árvore.



Hierarquia de diretórios



Hierarquia de diretórios organizada em árvore

Motivação (Por que utilizar?)

- O padrão *Composite* propõe que pastas e arquivos sejam tratados pelo cliente de maneira uniforme e transparente, evitando a utilização de operações condicionais (**if**) e teste de tipo de objeto (**instanceof**) feitos pelos clientes.
- Devemos criar uma classe que será comum aos arquivos (folhas) e pastas (nós), ou seja, eles terão o mesmo supertipo.
- Embora herdem características da mesma classe, os arquivos e pastas são diferentes. Um arquivo não possui filhos, portanto, para arquivos, **adicionar()**, **remover()** ou **recuperarFilho()** são métodos inadequados.

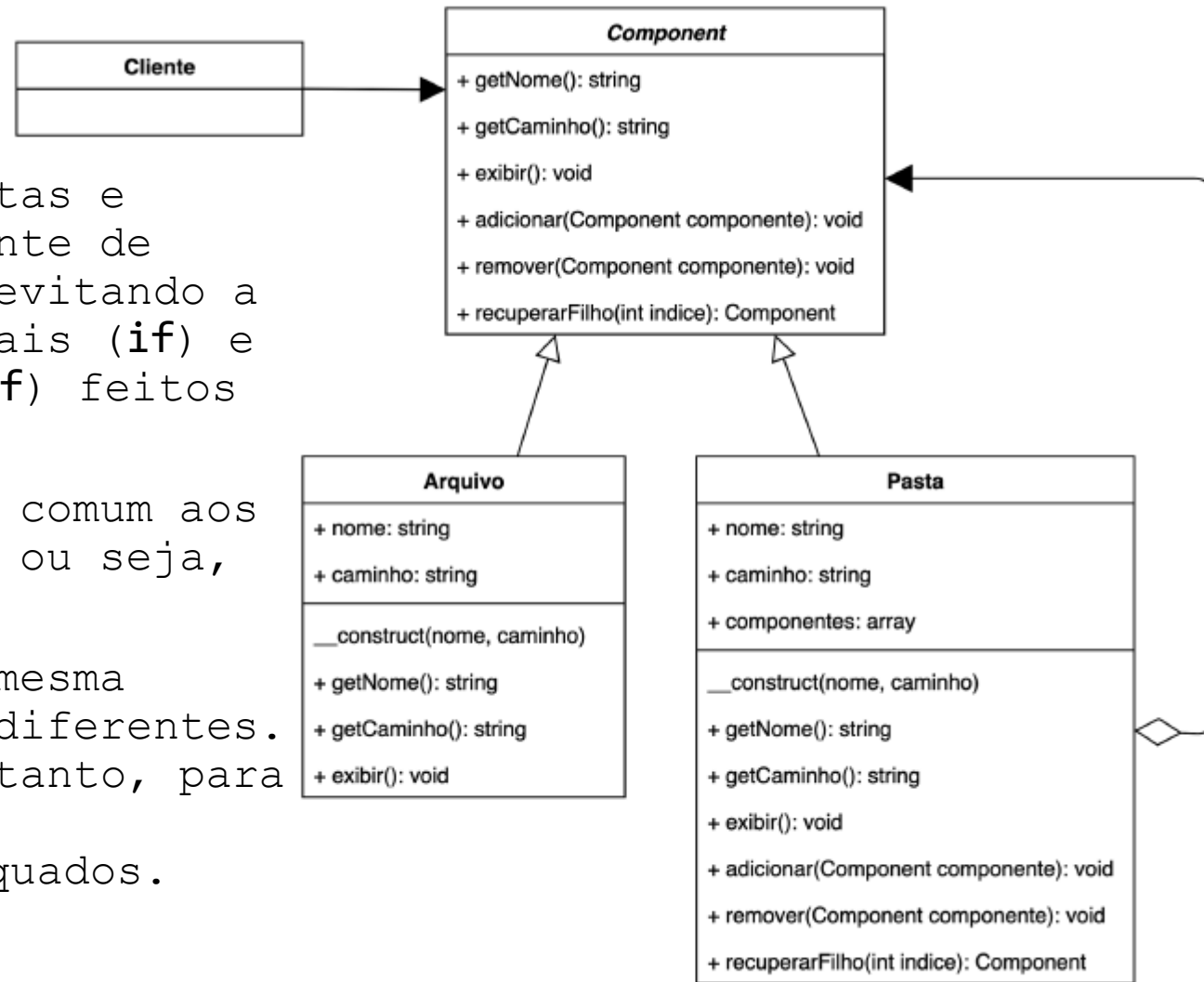


Diagrama de Classes do gerenciador de arquivos

Motivação (Por que utilizar?)

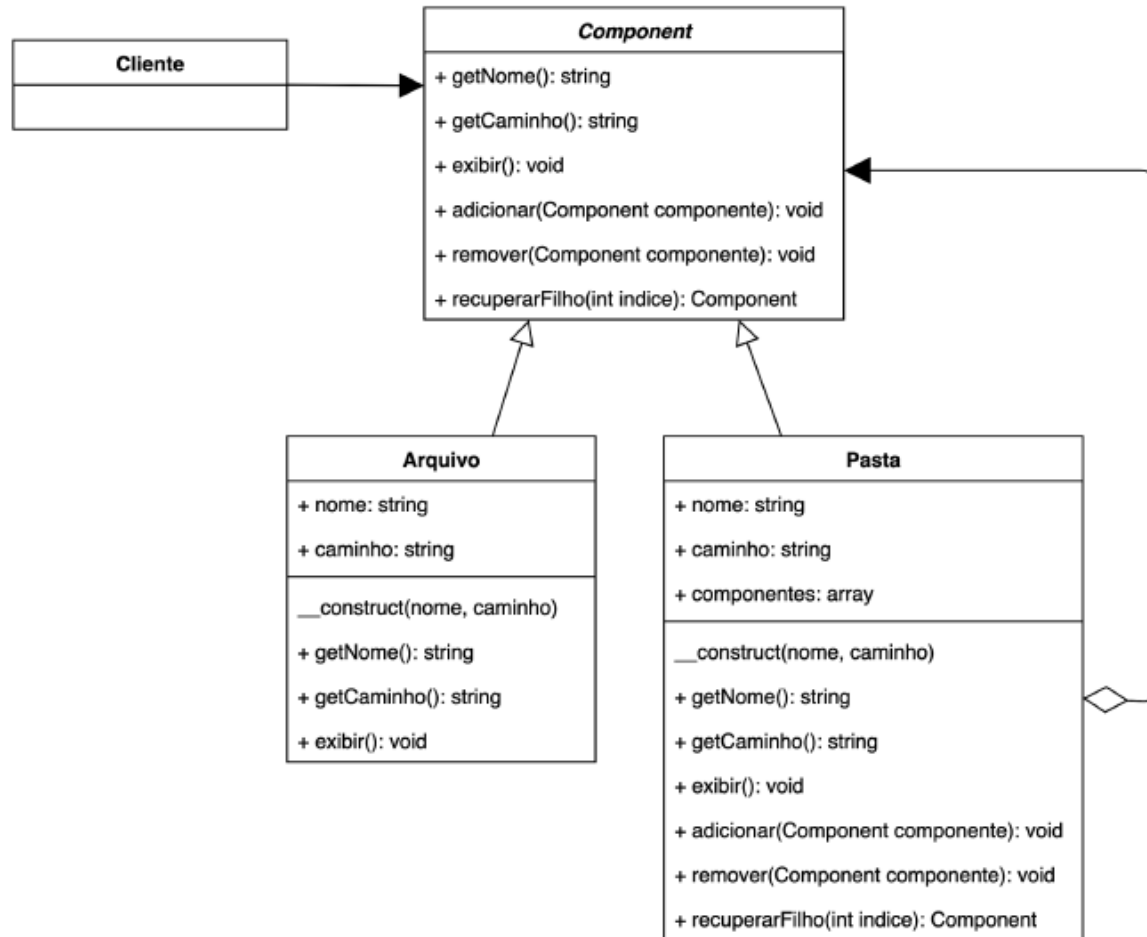


Diagrama de Classes do gerenciador de arquivos

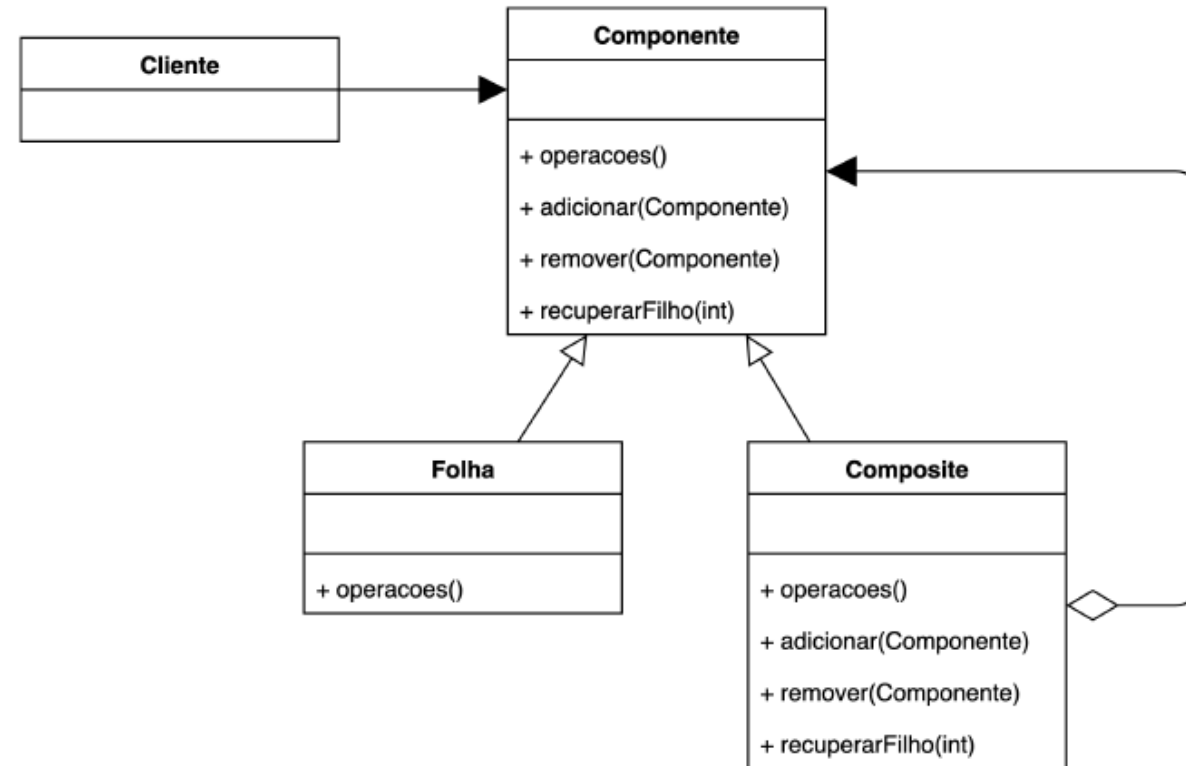


Diagrama de Classes

Motivação (Por que utilizar?)

- No diagrama de classes dos gerenciador de arquivos não existem métodos de **Component** que são inadequados a classe **Pasta**, pois mesmo sendo diferente de um arquivo, uma pasta ainda possui um nome, caminho e pode ser exibida. Porém, vale ressaltar que dependendo do contexto, podem existir métodos no **Component** que serão inadequados aos nós (**Pasta**) e/ou folhas (**Arquivo**).

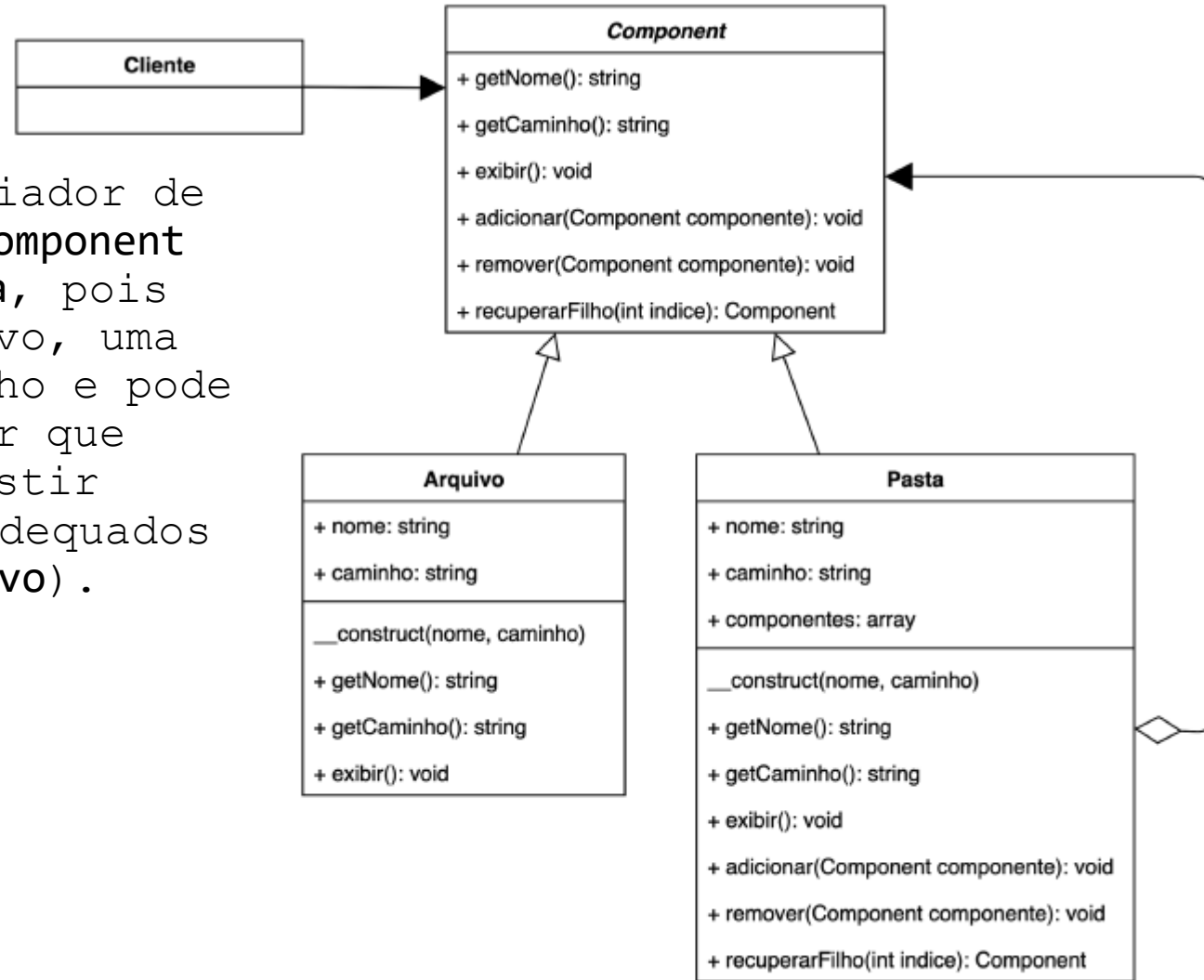
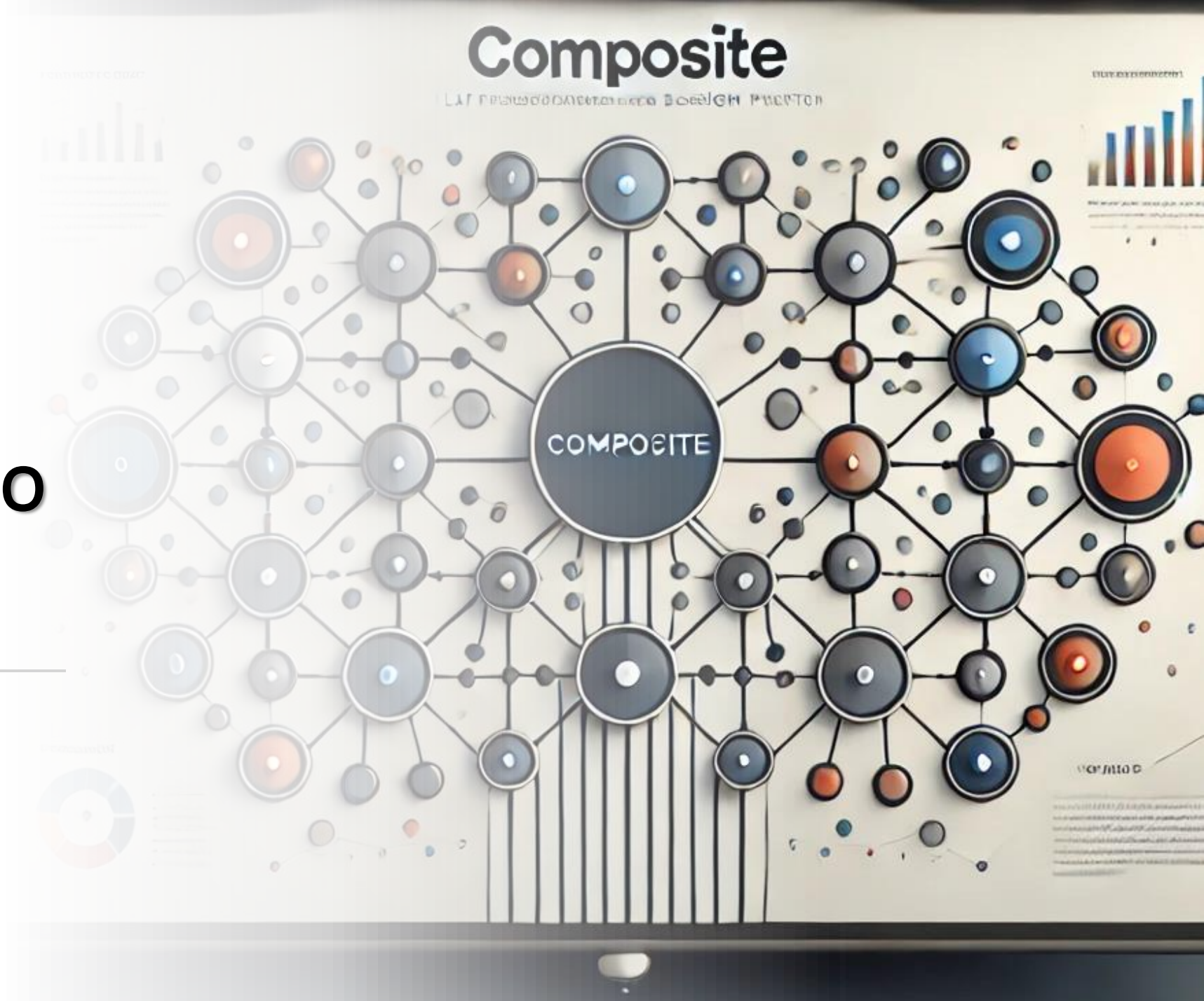


Diagrama de Classes do gerenciador de arquivos

Composite Implementação em C#

Padrões de Projeto Estrutural II

Prof. Me Jefferson Passerini



C#

- Vamos implementar as classes de acordo com o proposto com o design pattern Composite.
- **Primeiro vamos criar a classe abstrata Component.**

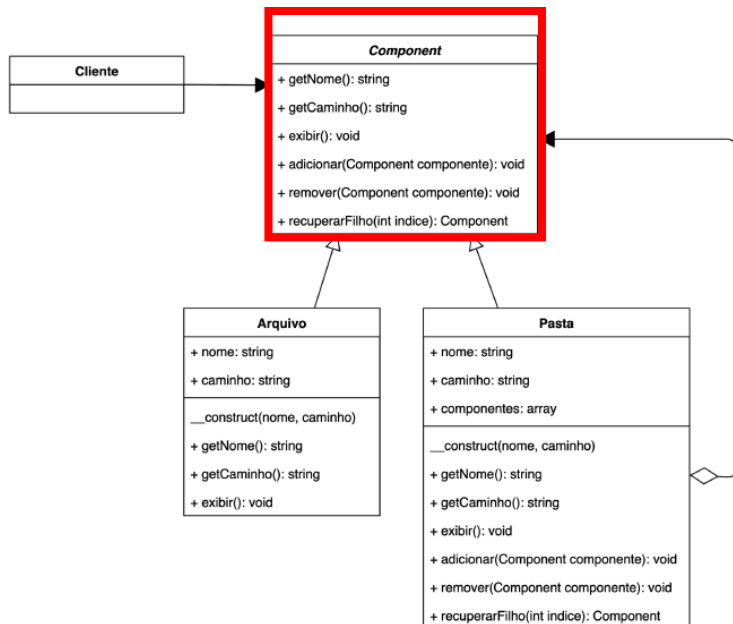
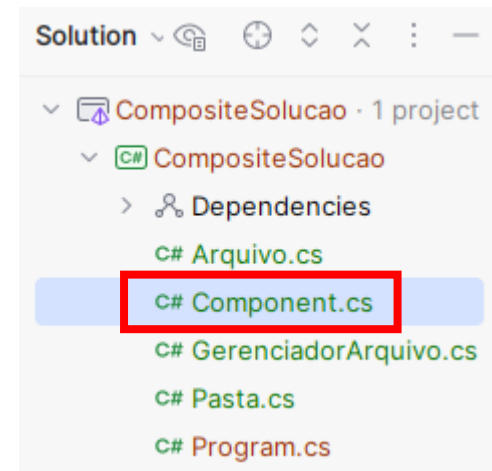


Diagrama de Classes do gerenciador de arquivos

```
1 {}  
2  
3 namespace CompositeSolucao;  
4  
5 public abstract class Component  
6 {  
7     10 usages 2 overrides  
8     public abstract void adicionar(Component componente);  
9     2 overrides  
10    public abstract void remover(Component componente);  
11    2 overrides  
12    public abstract Component recuperarFilho(int indice);  
13    2 usages 2 overrides  
14    public abstract void exibir();  
15 }  
16  
17 }
```



C#

- Implemente a classe concreta Pasta.

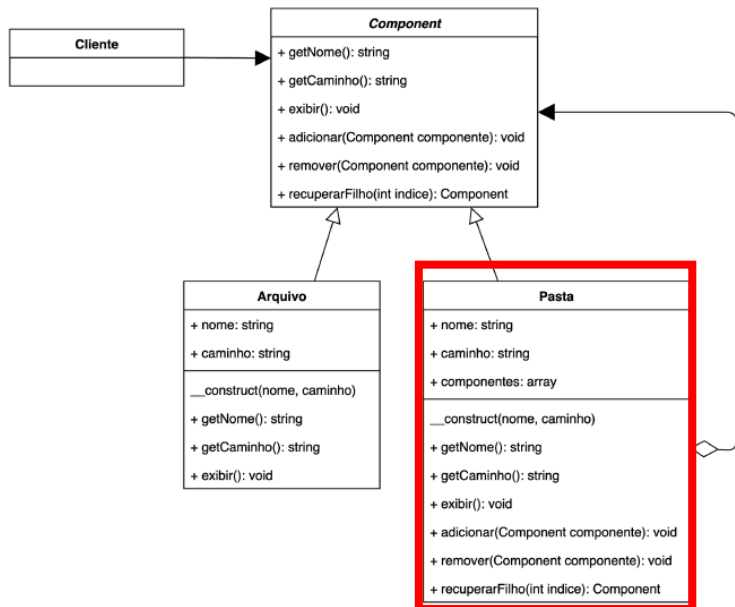


Diagrama de Classes do gerenciador de arquivos

```

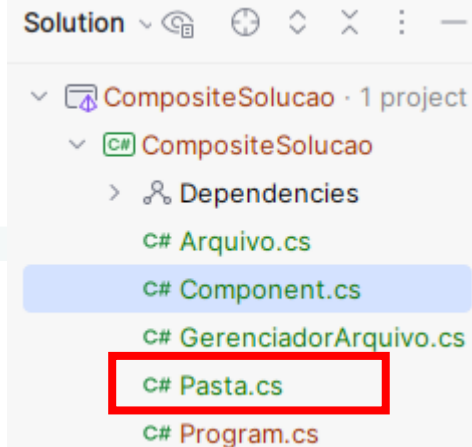
1 {} namespace CompositeSolucao;
2
3 ^o 5 usages
4 public class Pasta : Component
5 {
6     public String? nome;
7     public String? caminho;
8     public List<Component> componentes;
9
10    5 usages
11    public Pasta(string? nome, string? caminho)
12    {
13        this.nome = nome;
14        this.caminho = caminho;
15        this.componentes = new List<Component>();
16    }
17
18    0+10 usages
19    public override void adicionar(Component componente)
20    {
21        this.componentes.Add(componente);
22    }
23
24    public override void remover(Component componente)
25    {
26        foreach (var comp:Component in this.componentes)
27        {
28            if (comp == componente)
29            {
30                componentes.Remove(comp);
31            }
32        }
33    }
34 }

```

```

31
32 ^o
33
34 public override Component recuperarFilho(int indice)
35 {
36     return this.componentes[indice];
37 }
38
39 0+2 usages
40 public override void exibir()
41 {
42     String retorno = $" - {this.nome} - [{this.caminho}]";
43     Console.WriteLine("-----");
44     Console.WriteLine(retorno);
45
46     foreach (var comp:Component in this.componentes)
47     {
48         comp.exibir();
49     }
50 }
51
52 public String getNome()
53 {
54     return this.nome;
55 }
56
57 public String getCaminho()
58 {
59     return this.caminho;
60 }

```



C#

- Implemente a classe concreta Arquivo.

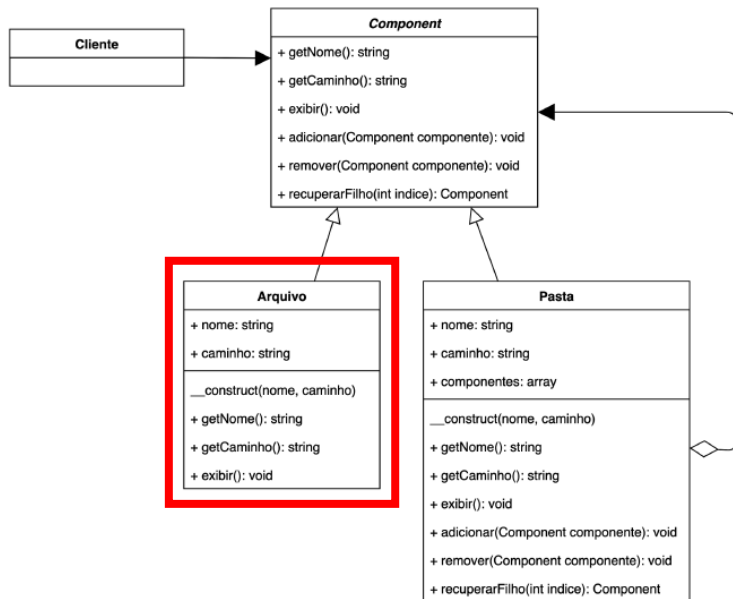
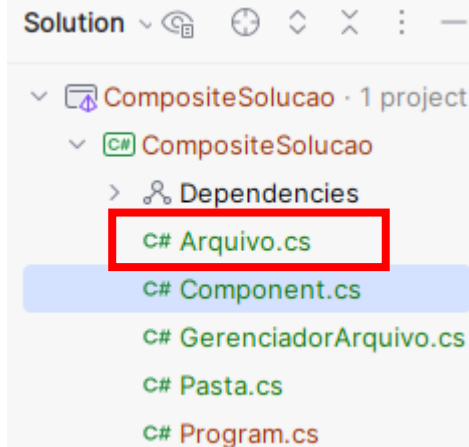


Diagrama de Classes do gerenciador de arquivos

```
1 {} namespace CompositeSolucao;
2
3 ^o public class Arquivo : Component
4 {
5     public String? nome;
6     public String? caminho;
7
8     public Arquivo(string? nome, string? caminho)
9     {
10         this.nome = nome;
11         this.caminho = caminho;
12     }
13
14     public String getNome()
15     {
16         return this.nome;
17     }
18
19     public String getCaminho()
20     {
21         return this.caminho;
22     }
23 }
```

```
24 ^o public override void exibir()
25 {
26     String retorno = $" - {this.nome} - [{this.caminho}]";
27     Console.WriteLine(retorno);
28 }
29
30 ^o public override void adicionar(Component componente)
31 {
32     throw new NotImplementedException();
33 }
34
35 ^o public override void remover(Component componente)
36 {
37     throw new NotImplementedException();
38 }
39
40 ^o public override Component recuperarFilho(int indice)
41 {
42     throw new NotImplementedException();
43 }
44 }
```



C#

- Implemente a classe concreta **GerenciadorSolucao**.

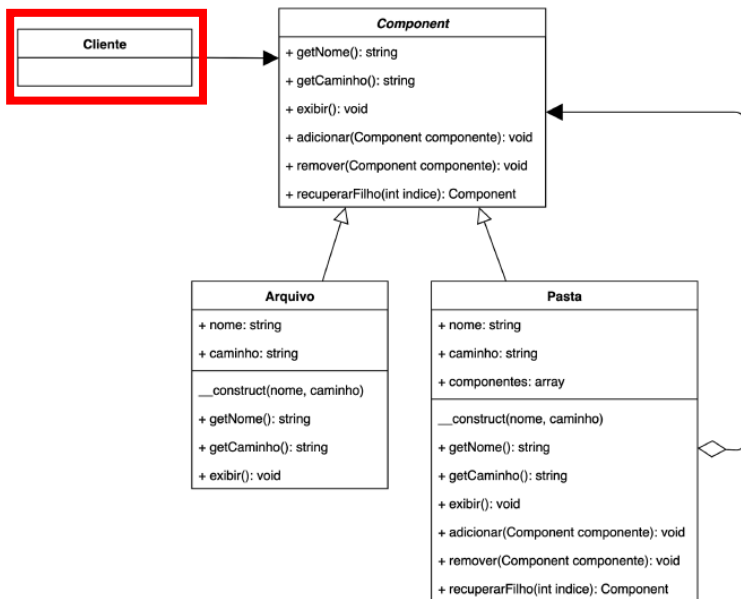
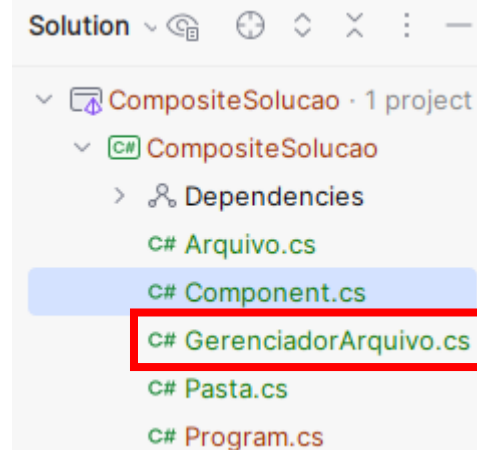


Diagrama de Classes do gerenciador de arquivos

```
1 {} namespace CompositeSolucao;
2
3 public class GerenciadorArquivo
4 {
5     private Component raiz;
6
7     public GerenciadorArquivo(Component raiz)
8     {
9         this.raiz = raiz;
10    }
11
12    public void exibirTodos()
13    {
14        this.raiz.exibir();
15    }
16 }
```



Program

```
1
2 {} using CompositeSolucao;
3
4 //cria pastas
5 Component raiz = new Pasta(nome: "Raiz", caminho: "/");
6 Component pasta1 = new Pasta(nome: "Pasta 1", caminho: "pasta1/");
7 Component pasta2 = new Pasta(nome: "Pasta 2", caminho: "pasta2/");
8 Component pasta3 = new Pasta(nome: "Pasta 3", caminho: "pasta3/");
9 Component pasta2_1 = new Pasta(nome: "Pasta 2.1", caminho: "pasta2_1/");
10
11 //Adiciona as pasta para montar a hierarquia
12 raiz.adicionar(pasta1);
13 raiz.adicionar(pasta2);
14 raiz.adicionar(pasta3);
15 pasta2.adicionar(pasta2_1);
16
17 //cria os arquivos
18 Component arquivo01 = new Arquivo(nome: "Arquivo01", caminho: "/arquivo01");
19 Component arquivo02 = new Arquivo(nome: "Arquivo02", caminho: "/arquivo02");
20 Component arquivo03 = new Arquivo(nome: "Arquivo03", caminho: "/arquivo03");
21 Component arquivo04 = new Arquivo(nome: "Arquivo04", caminho: "/arquivo04");
22 Component arquivo05 = new Arquivo(nome: "Arquivo05", caminho: "/arquivo05");
23 Component arquivo06 = new Arquivo(nome: "Arquivo06", caminho: "/arquivo06");
24
25 //adiciona os arquivos as pastas
26 pasta1.adicionar(arquivo01);
27 pasta1.adicionar(arquivo02);
28 pasta2.adicionar(arquivo03);
29 pasta2.adicionar(arquivo04);
30 pasta2_1.adicionar(arquivo05);
31 pasta3.adicionar(arquivo06);
32
33 //criacao do gerenciador de arquivos
34 GerenciadorArquivo gerenciador = new GerenciadorArquivo(raiz);
35 gerenciador.exibirTodos();
```

- Raiz - [/]

- Pasta 1 - [pasta1/]
- Arquivo01 - [/arquivo01]
- Arquivo02 - [/arquivo02]

- Pasta 2 - [pasta2/]

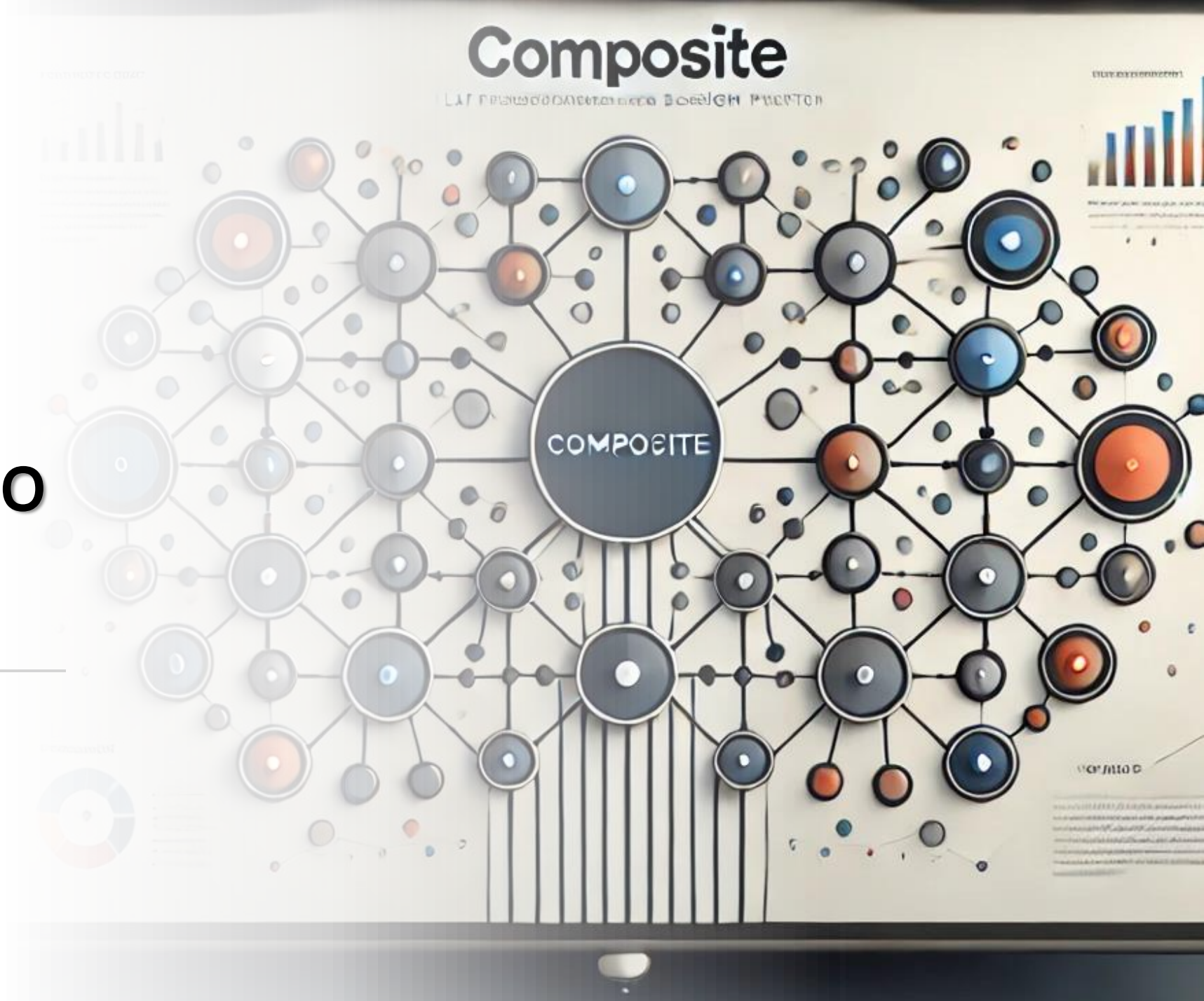
- Pasta 2.1 - [pasta2_1/]
- Arquivo05 - [/arquivo05]
- Arquivo03 - [/arquivo03]
- Arquivo04 - [/arquivo04]

- Pasta 3 - [pasta3/]
- Arquivo06 - [/arquivo06]

Composite Implementação em Java

Padrões de Projeto Estrutural II

Prof. Me Jefferson Passerini



Composite Consequências

Padrões de Projeto Estrutural II

Prof. Me Jefferson Passerini



Consequências

- Definição de hierarquias de classes que consistem em objetos primitivos (folhas) e objetos compostos (nós). Objetos primitivos podem compor objetos mais complexos, que por sua vez podem compor outros objetos e assim por diante recursivamente. Sempre que o cliente espera um objeto primitivo, ele também pode receber um objeto composto.
- Os clientes podem tratar estruturas compostas e objetos individuais de maneira uniforme. Normalmente, os clientes não sabem (e não devem se importar) se estão lidando com uma folha ou um componente composto.
- Facilita a adição de novos tipos de componentes. Os clientes não precisam ser alterados para novas classes de componentes sejam criadas.

Consequências

- **Pode tornar a arquitetura do código excessivamente genérica.** A desvantagem de facilitar a adição de novos componentes é a dificuldade de restrição dos tipos de componentes que um composto pode ter. Às vezes, é preciso que um composto tenha apenas determinados componentes. Com o *Composite*, não se pode confiar no sistema de tipos para impor essas restrições. Para isso será necessário realizar verificações em tempo de execução.
- **!importante:** Para contornar o problema citado acima, é possível separar a classe Componente em duas classes, uma para as folhas e outra para os Compostos (*Composite*) com seus respectivos métodos. Esta alternativa traz mais segurança ao código em relação à verificação dos tipos dos objetos. Por outro lado, a transparência por parte do cliente é perdida, já que assim que receber um objeto, ele terá que verificar se é um composto ou uma folha.
- A decisão de qual abordagem seguir vai de acordo com as necessidades e especificações de seu projeto. Se o mais importante é a transparência e baixa complexidade ou a segurança em relação aos tipos dos objetos.